

CS251A - Advanced Computer Architecture

HW1 - Microbench Analysis

Overall Analysis

This report analyzes the performance of various system configurations in gem5 using five benchmarks: *lfsr*, *merge*, *mm*, *sieve*, and *spmv*. The experiments modified different microarchitectural parameters, including clock frequency, issue width, CPU type, L2 cache size and compiler optimization levels, as summarized in Table 1.

Config Name	CPU	IssueWidth	Frequency	L2 Cache	Compiler
Baseline	OOO	8	2 GHz	2MB	-O3
Minor CPU	In-Order	8	2 GHz	2MB	-O3
Issue Width 2	OOO	2	2 GHz	2MB	-O3
Clk Freq 1GHz	OOO	8	1 GHz	2MB	-O3
Clk Freq 4GHz	OOO	8	4 GHz	2MB	-O3
No L2 Cache	OOO	8	2 GHz	-	-O3
256KB L2 Cache	OOO	8	2 GHz	256KB	-O3
16MB L2 Cache	OOO	8	2 GHz	16MB	-O3
-O1 Compile	OOO	8	2 GHz	2MB	-O1

Table 1 - Summary of System Configurations

What metric (and mean) should you use to compare the performance between different system configurations? Why?

The primary metric chosen for overall analysis is **IPC** (Instructions per Cycle) as it measures execution efficiency independent of clock speed. To understand the impact caused by clock frequency, **IPS** (Instructions per Second) is also considered. Additional metrics, such as **Cache Miss Rate**, **Cache Occupancy**, **OOO issue rate** and **CPU Idle Time**, are used to identify bottlenecks and understand the impact of each configuration change.

Impact of Different Configuration Changes:

a. CPU Type -

Change Tested: DerivO3CPU (OOO) → MinorCPU (In-Order)

- Out-of-order hides pipeline stalls caused by data dependencies, memory access latencies or control flow divergences. During in-order execution, the programs cannot utilize any form of instruction-level parallelism, and the IPC drops significantly.
- A large drop is seen in almost all the benchmarks, with ~64% in *merge* to ~86% in *mm*.

b. Issue Width -

Change Tested: Issue Width 8 → Issue Width 2

- Reducing the issue width limits parallel execution, causing a sharp IPC drop in compute-bound programs like *mm*, while memory-bound programs like *sieve* see less impact due to memory latency limitations.
- ~65% IPC drop is seen in *mm*, indicating compute-intensivity and only 27% drop in *sieve* suggests its memory-dependence.

IPC Comparison

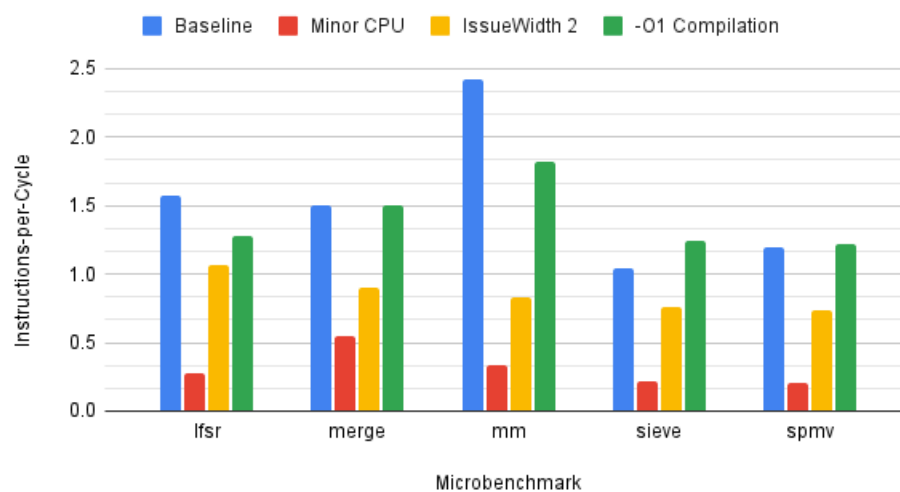


Fig 1 - IPC vs. CPU Type, Issue Width and Compiler Optimization

c. Compiler Optimization -

Change Tested: gcc -O3 → gcc -O1

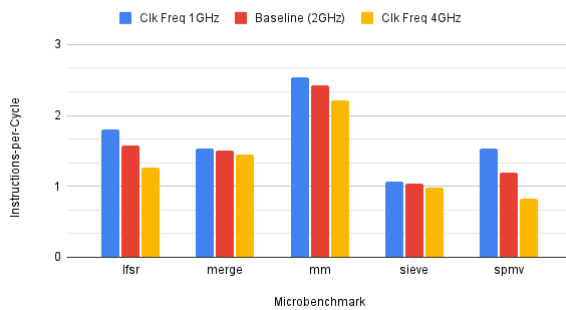
- Compiler optimization impacts vary by benchmark. Some benefit from the reduced optimization (*sieve*), others see minimal change (*spmv*, *merge*), and some perform worse (*mm*, *lfsr*). This entirely depends on program heuristics and cannot be generalized.

d. Clock Frequency -

Change Tested: 2GHz → 1GHz and 4GHz

- Lower clock frequencies efficiently mask memory latency, slightly increasing IPC but drastically reducing IPS. Higher clock frequencies reduce execution time but not necessarily improve IPC for memory-intensive programs due to frequent stalls.

IPC vs. Clock Frequency



IPS vs. Clock Frequency

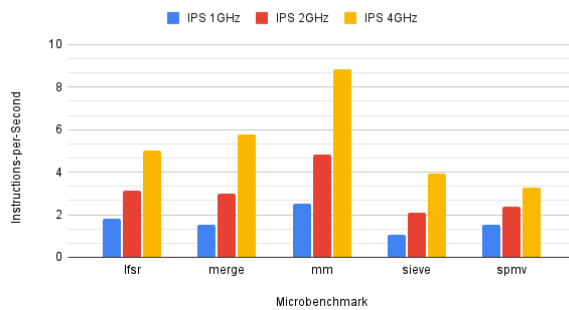


Fig 2 - Execution Efficiency vs. Clock Frequency

e. L2 Cache Size -

Change Tested: 2MB → No L2, 256KB, 16MB

- Larger L2 cache improves IPC for programs with frequently accessed data and good locality (*spm*), while programs with poor locality (*mm*) see little benefit. Removing the L2 cache unexpectedly boosts *sieve*'s IPC due to reduced memory lookup overhead. (Discussed later)

IPC vs. L2 Cache

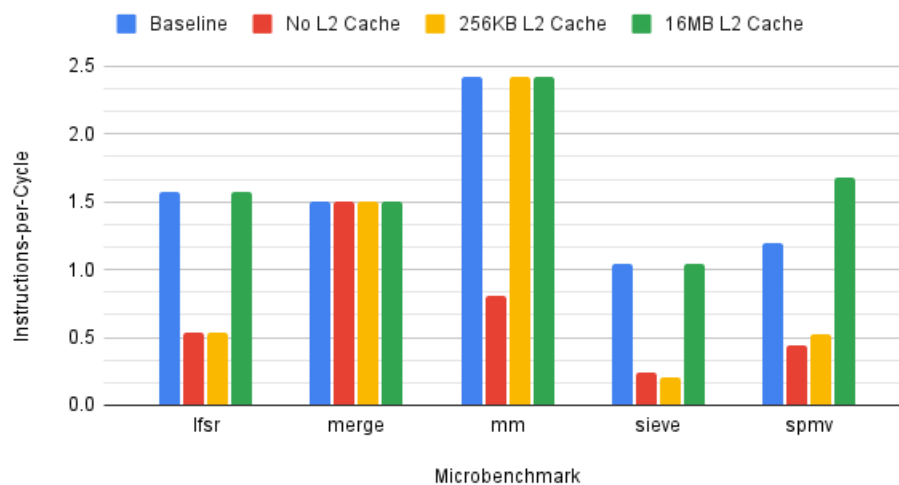


Fig 3 - IPC vs. L2 Cache Size

What's more important, high issue width or OOO execution? Does it depend on the application?

IPC drops drastically with an In-Order CPU core for all the programs, indicating that these programs use dynamic instruction scheduling to maintain performance. Reducing issue width reduction also causes the IPC to drop, but the impact is less severe than switching to in-order.

Program	idleCycles			issueRate	
	Baseline	Minor CPU	IssueWidth 2	Baseline (IssueWidth 8)	IssueWidth 2
lfsr	26889	10133229	34334	2.500031	1.695032
merge	29332	109736	35538	3.591775	1.609546
mm	28774	369898	34934	5.55665	1.912844
sieve	29937	307123	36281	1.867477	1.356742
spmv	28378	15115105	34704	2.426885	1.472186

Table 2 - Stats to compare OOO with Issue Width

- Reducing the Issue Width to 2 lowers the issue rate significantly, but idle cycles increase only slightly compared to in-order execution. *mm* issue rate drops from 5.5 to 1.9, demonstrating its high ILP and compute-bound nature.
- Memory-bound workloads like *spmv* and control-heavy benchmarks like *merge* do not benefit much from wider issue width, and need OOO execution to reduce stalls.

OOO execution is generally more important than issue width, unless the workload has very high ILP where a wider issue width can further boost performance.

Did any benchmarks benefit from removing the L2 cache, why or why not?

sieve benefits slightly from removing the L2 cache (16% increase in IPC from 256KB L2 cache).

Metric	No L2 Cache	256KB L2 Cache	Analysis
L1 Cache Miss Rate	0.344808	0.288087	L1 hit rate is slightly worse without L2, but not significantly.
L1 Cache Occupancy	0.998343	0.998537	L1 is fully occupied in both cases.
L2 Cache Miss Rate	–	0.868031	L2 is fully occupied, but ineffective due to such a poor hit rate.
L2 Cache Occupancy	–	0.996903	

Table 3 - Cache Metrics for *sieve* benchmark

With 256KB L2 cache, almost 87% of L2 accesses miss, meaning that most requests eventually go to DRAM anyway, adding unnecessary L2 lookup overhead. The L1 cache is always occupied, holding the useful data. The slight increase in L1 misses is compensated by faster overall memory access time, boosting performance.

Microbenchmark Analysis

For each microbenchmark:

- a. Describe/characterize its: a) memory regularity, b) control regularity, c) Locality*
- b. explain which microarchitecture parameter is it most sensitive to (in other words, what do you think the "bottleneck" is), and justify using reasoning or statistics.*

lfsr

- **Memory Regularity:** LFSR generates random memory accesses to a large array. The low L2 miss rate (7%) and high L2 occupancy (87%) depicts that data is accessed somewhat predictably, and the cache is effective in capturing reuse.
- **Control Regularity:** There is minimal control flow, with a single loop iterating over random indices. OOO execution significantly improves performance (IPC drops 1.576 -> 0.276 with in-order), indicating that OOO helps resolve data dependencies.
- **Locality:** High L1 Miss rate (97%) and low L2 miss rate (7%) suggest that L2 cache is crucial for compensating frequent L1 misses. Remove L2 cache drops IPC by 66% confirming its importance.
- **Bottleneck:** The high L1 miss rate and reliance on L2 cache indicate that memory latency is the primary bottleneck.

merge

- **Memory Regularity:** merge has unpredictable memory access patterns as seen in the recursive calls and merging steps. The high L2 miss rate (75%) and low L2 occupancy (4%) confirm this irregularity.
- **Control Regularity:** The recursive calls and conditional statements cause frequent branching. In-order execution causes IPC drop and increased idle time (5x), suggesting frequent stalls due to branching.
- **Locality:** The low cache occupancy and poor data reuse suggest that this program does not benefit much from caching. Cache sizes have minimal impact on IPC.
- **Bottleneck:** Frequent branching and poor data reuse make branch prediction and memory latency as the bottlenecks.

mm

- **Memory Regularity:** mm accesses memory in regular strided patterns due to its block-based computation. Low L2 miss rate (6%) and low L2 occupancy (8%) show its efficient data reuse.
- **Control Regularity:** There is minimal control flow divergence with nested loop iterations. Reducing the issue width causes a major IPC drop (2.4 to 0.8), denoting high parallelism.
- **Locality:** High L1 occupancy (95%) suggests effective data reuse within the L1 cache. Removing the L2 cache drops IPC by 66% because of reliance on two-level cache.
- **Bottleneck:** Its compute-intensive nature makes issue width the primary bottleneck.

sieve

- **Memory Regularity:** Moderate L2 occupancy (47%) and miss rate (25%) suggest some cache reuse. The memory access pattern is random due to prime numbers.
- **Control Regularity:** There is minimal branching. During in-order execution, IPC drops by 80% and idle cycles increase by 10x, suggesting high instruction-level parallelism.
- **Locality:** L1 cache is sufficient (99% occupancy), and the L2 cache is ineffective and does not promote data reuse. Removing L2 slightly boosts IPC, meaning that L2 holds useless data and adds to memory lookup time.
- **Bottleneck:** The random memory access patterns makes memory latency the primary bottleneck.

spmv

- **Memory Regularity:** Irregular and frequent memory accesses are suggested by 92% L2 cache occupancy, and moderate miss rate ~15%.
- **Control Regularity:** There is minimal control flow with nested loop iterations. IPC drops significantly during in-order execution (1.1 to 0.2) while idle cycles skyrocket (28K to 15M) indicating that OOO is essential to mask stalls caused by data dependencies.
- **Locality:** Large L2 caches boost performance, meaning efficient data reuse. High L1 miss rate (34%) confirms poor temporal locality.
- **Bottleneck:** Irregular memory access patterns and frequent stalls make memory latency the primary bottleneck.

Enhancements

Pick a microbenchmark besides matrix multiply; propose one application enhancement, ISA enhancement, and microarchitecture enhancement that you think would be very effective for that microbenchmark.

lfsr is chosen for optimization, because it has minimal control flow complexity and high cache locality, but suffers from slower execution due to time-consuming arithmetic operations.

Application Enhancement: Loop Unrolling

lfsr performs a large number of iterations and each involves multiple bitwise operations. Loop iterations can be expanded manually to allow for better parallelization, reducing overall execution time. Loop unrolling can further improve cache utilization by reducing the number of instruction fetches and saving useful computations per iteration.

Example:

```
int unroll_factor = 4; // Unrolling loop by 4
int bound = ITERS - (ITERS % unroll_factor); // remainder iterations

// Perform 4 LFSR iterations in a single loop
```

```

while (count < bound) {
    count += unroll_factor;

    lfsr = (lfsr >> 1) ^ (-(lfsr & 1u) & 0xB400u);
    result += arr[lfsr].p1;

    lfsr = (lfsr >> 1) ^ (-(lfsr & 1u) & 0xB400u);
    result += arr[lfsr].p1;

    lfsr = (lfsr >> 1) ^ (-(lfsr & 1u) & 0xB400u);
    result += arr[lfsr].p1;

    lfsr = (lfsr >> 1) ^ (-(lfsr & 1u) & 0xB400u);
    result += arr[lfsr].p1;

}

// Handle remaining iterations
while (count < ITERS) {
    count++;
    lfsr = (lfsr >> 1) ^ (-(lfsr & 1u) & 0xB400u);
    result += arr[lfsr].p1;
}

```

ISA Enhancement: Dedicated Instruction

lfsr operations are essentially bitwise XOR, AND and shift operations, each of which requires multiple instructions per iteration. A new instruction for LFSR (ex LFSR-STEP) can be introduced to automatically shift bits and apply XOR in a single cycle, reducing instruction count and improving throughput. The feedback polynomial can be a configurable input.

Example:

```
LFSR-STEP rd, rs1, rs2
```

rd: Destination register (stores new LFSR state)

rs1: Source register (current LFSR state)

rs2: Feedback polynomial (pre-defined mask for LFSR taps ex. 0xB400)

Microarchitecture Enhancement:

lfsr execution is a mix of multiple bitwise operations (shift, AND, XOR), which are inefficient on general-purpose ALUs. Moreover, this instruction mix causes pipeline stalls due to data dependencies. A new execution unit dedicated to LFSR can be implemented that directly computes the next LFSR state. The CPU would offload LFSR computation tasks to this new unit on encountering LFSR-STEP instruction, freeing ALU for other tasks.